

Google Assistant - IM Report

Dept. de Eletrónica, Telecomunicações e Informática
Universidade de Aveiro Universidade de Aveiro



Duarte Santos, Henrique Teixeira
(113304) duartesantos@ua.pt, (114588) henriqueft04@ua.pt

Conteúdo

1	Introdução	1
2	Arquitetura	2
2.1	Visão Geral	2
2.2	Componentes do Sistema	2
2.3	Fluxo de Arquitetura	3
3	Requisitos do Sistema	4
3.1	Requisitos de Hardware	4
3.2	Requisitos de Software	4
3.3	Bibliotecas e Frameworks Python	4
3.4	Componentes do Framework Multimodal	5
4	Arquitetura de Fusão Multimodal	6
4.1	Conceitos Fundamentais de Fusão	6
4.2	Implementação de Complementaridade	6
4.3	Fluxo Completo de Fusão	7
4.4	Melhorias e Otimizações Implementadas	8
5	Modalidades de Interação	13
5.1	Modalidade de Voz com RASA NLU	13
5.2	Modalidade de Saída: Text-to-Speech	16
6	Conclusões	17
6.1	Síntese do Trabalho Realizado	17
6.2	Cumprimento de Objetivos	17
6.3	Dificuldades Encontradas e Soluções	18
6.4	Reflexão Final	20

Capítulo 1

Introdução

O sistema desenvolvido consiste num assistente multimodal que controla o Google Maps por voz e/ou por gestos por um utilizador. De forma a garantir uma interação fluida, o assistente fornece feedback auditivo a cada ação executada pelo utilizador. O projeto foi implementado na linguagem Python, com recurso à biblioteca Selenium. A escolha desta tecnologia deve-se à capacidade do Selenium automatizar e controlar o browsing na web.

Para os componentes de interação (voz e gestos) e processamento central (Motor de Fusão), foi adotada a framework multimodal do IEETA.

O projeto foi desenvolvido em três iterações distintas:

- **Iteração 1 - Voz:** Implementação de um assistente controlado por voz utilizando RASA NLU para reconhecimento de intenções;
- **Iteração 2 - Gestos:** Adição de reconhecimento de gestos através de Kinect e GenericGesturesModality;
- **Iteração 3 - Fusão:** Integração das duas modalidades (voz e gestos) através de um motor de fusão baseado em SCXML. Esta iteração permite ao utilizador interagir com o Google Maps usando voz, gestos ou ambos simultaneamente.

Capítulo 2

Arquitetura

2.1 Visão Geral

A arquitetura final descreve um sistema de interação multimodal que permite aos utilizadores controlar a interface web do Google Maps através da combinação natural de comandos de voz e gestos. Utiliza o protocolo **MMI** (MultiModal Interaction) para fazer a comunicação entre múltiplos componentes. O objetivo principal é permitir controlar a interface web (Google Maps) através da fusão de comandos de voz (Microfone) e gestos (Kinect), utilizando um modelo de integração centrado num motor de fusão.

2.2 Componentes do Sistema

A arquitetura pode ser dividida em quatro principais camadas que refletem o fluxo natural de processamento de informação: desde a captura de inputs do utilizador (percepção), passando pela interpretação e combinação desses inputs (fusão), seguida pela execução de ações (lógica de aplicação), e terminando com a apresentação de resultados ao utilizador (feedback).

Nas subsecções seguintes, cada camada será descrita em detalhe, explicando os componentes em si bem como a função no sistema.

2.2.1 Input Modalities

A parte referente à modalidade gestual, ou seja, à captura de movimento é realizada pelo sensor Kinect. No fundo, os dados são processados pelo componente **GenericGesturesModality**, responsável por identificar os gestos do utilizador e enviá-los para o sistema de fusão.

Por outro lado, a modalidade vocal, reconhecimento de voz, é processado pelo **Rasa NLU**. Este componente recebe o texto, após reconhecimento de fala, e extrai as intenções semânticas e entidades, enviando eventos estruturados para o sistema.

2.2.2 Fusion & Management

O sistema de fusão é composto por múltiplos componentes que comunicam através do protocolo MMI (MultiModal Interaction). O sistema inclui um **FusionEngine** baseado em SCXML que recebe eventos das duas modalidades: (1) comandos de voz, e (2) gestos. No fundo, ele encaminha esses eventos para o **Python Assistant** através da framework **MMI**.

2.2.3 Lógica de Aplicação

O componente que atua como servidor dentral de lógica é o **Python Assistante**, que tem como função receber os comandos previamente fundidos pelo FusionEngine e traduzir os tais comandos em ações concretas. Após o nosso assistente fazer a tradução passa a responsabilidade para o Selenium web driver, que é este o responsável pela manipulação diretamente na interface web do Google Maps, executados as ações ordenadas pelo utilizador.

2.2.4 Feedback

Como ultimo componente temos a camada de feedback onde está um serviço a rodar na porta 8083, o **servidor TTS**, que é responsável por gerar o feedback para o utilizador. Este servidor que comunica com uma **WebApp** baseada em browser, utilizando a Web Speech API para síntese de voz em português

2.3 Fluxo de Arquitetura

Esta secção descreve o fluxo completo de processamento de dados no sistema, desde a captura de input multimodal até à execução de ações e geração de feedback ao utilizador.

O fluxo inicia-se com a captura simultânea de duas modalidades de input, sendo elas:

- **Input Gestual:** Usa o sensor Kinect que captura os movimentos do utilizador. Os dados são enviados para o **Gesture Builder**, que processa e interpreta os gestos. Os gestos reconhecidos são então enviados para o **Generic Gestures Modality**.
- **Input Vocal:** Usa o microfone que faz captura a voz do utilizador. Após o reconhecimento da fala, o texto resultante é processado pelo **Rasa NLU**, que realiza a análise de linguagem natural para extrair resultados.

Após esta fase de captura, os eventos provenientes das duas modalidades então convergem para o **Fusion Engine**, o componente central da arquitetura, que é implementado em SCXML e é responsável por realizar a fusão dos comandos recolhidos e gerar comandos fundidos que são enviados para o **MMI**.

De seguida temos o componente **MMI** que no fundo atua como middleware de comunicação, recebendo os comandos previamente fundidos e envia-os para o **Assistant**. É neste mesmo **Assistant** onde os comandos são mapeados para as ações concretas na interface web do Google Maps. Esta manipulação é feita usando o **Selenium**.

Como última fase da nossa arquitetura temos a fase de feedback multimodal ao utilizador. Para parte visual, o utilizador consegue ter feedback visual imediatamente através das alterações na interface web do Google Maps. Já para o feedback sonoro, o **Assistant** envia mensagens de confirmação e informação para o **servidor TTS**.

Este fluxo contínuo permite uma interação natural e fluida, onde o utilizador pode combinar as interações de gestos e comandos de voz para controlar o Google Maps, recebendo feedback imediato das suas ações tanto visual como auditivamente. Na Figura 2.1 conseguimos ver de uma modo geral as interações entre os principais componentes do nosso sistema.

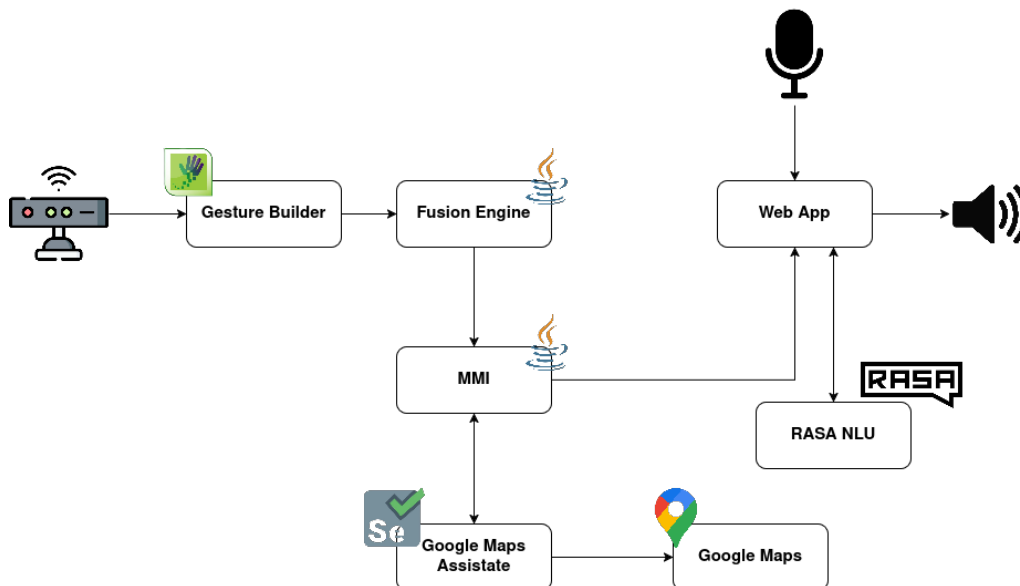


Figura 2.1: Arquitetura de Fusão

Capítulo 3

Requisitos do Sistema

O desenvolvimento e execução do sistema multimodal para controlo de mapas digitais requer um conjunto de componentes de hardware e software que garantam o correto funcionamento de todas as funcionalidades. Este capítulo apresenta os requisitos técnicos necessários para a instalação e operação do sistema.

3.1 Requisitos de Hardware

Para a execução adequada do sistema, é necessário dispor de equipamento capaz de suportar a captura de gestos e o processamento simultâneo de múltiplos componentes. Os requisitos de hardware incluem:

- Sensor **Microsoft Kinect v2** para captura de movimentos e gestos. O sensor Kinect é essencial para a modalidade gestual do sistema, sendo responsável pela captura de movimentos corporais que são posteriormente processados e convertidos em comandos. ;
- Conexão estável à **Internet** para acesso ao Google Maps.

3.2 Requisitos de Software

O sistema foi desenvolvido para operar em ambiente Windows, aproveitando o suporte nativo do SDK do Kinect para esta plataforma. Os requisitos de software base incluem:

- Sistema operativo **Windows 10** (64-bit);
- **Python** versão 3.10 ou superior;
- **Java JDK** versão 8 ou superior;
- **Google Chrome** ou Microsoft Edge (automação Selenium);
- **Kinect for Windows SDK v2**;

3.3 Bibliotecas e Frameworks Python

O sistema utiliza diversas bibliotecas Python. As principais dependências incluem:

- **RASA** versão 3.5.x para processamento de linguagem natural
- **Selenium** versão 4.15 ou superior para automação do browser
- **WebSockets** versão 12.0 ou superior para comunicação assíncrona
- **AsyncIO** para suporte de programação assíncrona (incluído no Python 3.10)
- **Colorlog** para sistema de logging estruturado

Todas as dependências Python estão especificadas no ficheiro `requirements.txt` incluído no projeto.

3.4 Componentes do Framework Multimodal

O sistema integra componentes da framework, responsáveis pela gestão de eventos e fusão de modalidades. Os componentes necessários são:

- **MMI Framework** - implementação do protocolo W3C para comunicação multimodal
- **FusionEngine** - motor SCXML para fusão de eventos de diferentes modalidades
- **GenericGesturesModality** - módulo de processamento de gestos do Kinect
- **Keystores SSL** (keystore2.jks) para comunicação segura HTTPS/WSS

Estes componentes Java comunicam entre si através do protocolo MMI, garantindo sincronização temporal e fusão adequada de comandos vocais e gestuais. A configuração destes componentes é realizada através de ficheiros XML específicos incluídos no projeto.

Capítulo 4

Arquitetura de Fusão Multimodal

Este capítulo detalha a implementação do motor de fusão multimodal, componente central que permite ao sistema combinar informação proveniente das modalidades de voz e gestos. O motor de fusão implementa diferentes estratégias de integração modal, permitindo que o utilizador interaja com o sistema de forma natural e flexível, utilizando uma ou ambas as modalidades conforme a tarefa e contexto.

4.1 Conceitos Fundamentais de Fusão

A fusão multimodal é o processo de combinar informação proveniente de múltiplas modalidades de input para gerar comandos unificados que são enviados para a aplicação, no âmbito deste projeto era a fusão entre as modalidades de interação de gestos e voz:

- **Modalidade de Voz:** Comandos vindos do RASA NLU, contendo intents e entidades extraídas de linguagem natural;
- **Modalidade de Gestos:** Comandos recolhidos pelo Kinect v2 através do GenericGestures-Modality, representando gestos físicos do utilizador.

4.2 Implementação de Complementaridade

A fusão complementar ocorre quando as duas modalidades fornecem informação distinta que, quando combinada.

O sistema implementa 8 combinações complementares que permitem ao utilizador controlar simultaneamente zoom e direção de navegação. Estas combinações cobrem todas as possibilidades de zoom (in/out) com as quatro direções de movimento (up/down/left/right). A Tabela 4.1 indica as possíveis formas de combinação de comandos de voz com os gestos gerando apenas um comando de saída.

4.2.1 Combinações Implementadas

Tabela 4.1: Comandos de fusão complementar implementados

Comando de Voz	Gesto	Comando Fundido
Redundância		
Aproximar	Zoom In (expandir braços)	ZOOM_IN
Afastar	Zoom Out (expandir braços)	ZOOM_OUT
Cancelar	Exit Street	CANCEL
Fusão Complementar - Zoom + Direções		
Aproximar (ZOOM_IN)	Swipe Up	ZOOM_IN_UP
Aproximar (ZOOM_IN)	Swipe Down	ZOOM_IN_DOWN
Aproximar (ZOOM_IN)	Swipe Left	ZOOM_IN_LEFT
Aproximar (ZOOM_IN)	Swipe Right	ZOOM_IN_RIGHT
Afastar (ZOOM_OUT)	Swipe Up	ZOOM_OUT_UP
Afastar (ZOOM_OUT)	Swipe Down	ZOOM_OUT_DOWN
Afastar (ZOOM_OUT)	Swipe Left	ZOOM_OUT_LEFT

Afastar (ZOOM_OUT)	Swipe Right	ZOOM_OUT_RIGHT
Obter direções (GET_DIRECTIONS)	Transports (gesto)	DIRECTIONS_PUBLIC_TRANSPORT
Mostrar tráfego (SHOW_TRAFFIC)	Zoom Out (gesto)	TRAFFIC_OVERVIEW
Restaurants (gesto)	Centrar mapa (RECENTER_MAP)	FILTER_RESTAURANTS_CENTER
Forward (gesto)	Iniciar navegação (START_NAVIGATION)	STREET_VIEW_FORWARD_CONTINUOUS

A fusão redundante ocorre quando ambas as modalidades expressam a mesma intenção, permitindo ao utilizador escolher a forma de interação mais conveniente para cada situação. Este paradigma aumenta a robustez do sistema.

A fusão redundante permite que o utilizador escolha livremente entre modalidades e possa executar um comando com essa modalidade escolhida. No fundo, o **FusionEngine** reconhece a redundância e gere um único evento fundido ambas as modalidades, e desta forma, apesar de para a mesma ação haver possibilidade de ambas as modalidades, o **Assistant** executa a ação apenas uma vez. O sistema implementa redundância para três comandos críticos.

4.2.2 Comandos Redundantes Implementados

Tabela 4.2: Comandos de fusão redundante implementados

Comando de Voz	Gesto	Resultado
Aproximar	Zoom In (expandir braços)	ZOOM_IN
Afastar	Zoom Out (expandir braços)	ZOOM_OUT
Cancelar	Exit Street	CANCEL

4.3 Fluxo Completo de Fusão

O processamento completo de um comando multimodal segue o seguinte fluxo:

1. **Captura:** Modalidades capturam input do utilizador (voz via microfone, gestos via Kinect)
2. **Reconhecimento:**
 - Voz processada por Speech-to-Text + RASA NLU
 - Gestos processados por GenericGesturesModality
3. **Envio MMI:** Cada modalidade envia evento MMI para FusionEngine (porta 8001):
 - Voz: POST /IM/USER1/SPEECHIN
 - Gestos: POST /IM/USER1/GESTURES
4. **Fusão SCXML:** FusionEngine processa eventos:
 - Armazena primeiro evento e inicia janela temporal
 - Se segundo evento chegar, funde modalidades
 - Se timeout, processa evento isoladamente
5. **Envio para Assistant:** FusionEngine envia evento fundido via MMI Server (porta 8005)
6. **Extração NLU:** Assistant extrai intent através de `nlu_extractor()`:

```
if command_type == "FUSION":
    fusion_intent = recognized[1] # e.g., "ZOOM_IN_UP"
    intent = fusion_intent
```

7. **Routing:** IntentRouter encaminha para handler apropriado
8. **Execução:** Handler executa ações no Google Maps via Selenium
9. **Feedback:** Sistema fornece feedback auditivo via TTS

4.3.1 Exemplo Completo: Fusão Complementar

Cenário: Utilizador diz "aproximar" e faz swipe up simultaneamente.

1. Web Speech API captura "aproximar" → RASA extrai intent `zoom_in`
2. WebApp envia `[SPEECH] [ZOOM_IN]` para FusionEngine
3. FusionEngine entra em estado `sspeech[zoom_in]`
4. Kinect deteta swipe up → GenericGesturesModality envia `[GESTURES] [SWIPE_UP]`
5. FusionEngine deteta transição complementar, cancela timer
6. Estado de fusão combina `data1` (voz) + `data2` (gesto)
7. FusionEngine envia `[FUSION] [ZOOM_IN_UP]` para Assistant
8. Assistant identifica comando fundido, invoca `ZoomInUpHandler`
9. Handler executa: `zoom_in(2)` → pausa 300ms → `pan_map("up", 3)`
10. TTS anuncia: "A aproximar e mover para cima"

4.4 Melhorias e Otimizações Implementadas

Durante a fusão e integração dos dois projetos (sistema multimodal de reconhecimento de voz/gestos e controlo automatizado do Google Maps), foram implementadas diversas melhorias significativas a nível de lógica, treino de modelos, tratamento de erros, desempenho e experiência de utilizador. Esta secção detalha as principais otimizações realizadas.

4.4.1 Melhorias no Processamento de Linguagem Natural

Expansão do Corpus de Treino RASA

O modelo RASA foi significativamente melhorado através da expansão do corpus de treino:

- **Aumento de Exemplos:** Adição de mais de 100 novos exemplos de treino para intents críticos, com foco em variações linguísticas do português europeu coloquial
- **Modos de Transporte:** Expansão massiva do intent `change_transport_mode` para suportar formas isoladas:
 - Palavras isoladas: "pé", "bicicleta", "carro", "autocarro", "comboio", "metro"
 - Formas verbais: "caminhar", "andar", "conduzir"
 - Variações compostas: "ir a pé", "ir de bicicleta", "ir de carro"
 - Comandos completos: "mudar para transportes públicos", "alterar modo de transporte"

Mapeamento Semântico de Entidades

Criação de sistema robusto de mapeamento para entidades de transporte em `enums.py`:

- **Mapeamento Bilingue:** Suporte para termos em inglês e português:
 - `TransportMode.DRIVING`: "driving", "drive", "car", "carro", "conduzir", "automóvel", "viatura"
 - `TransportMode.WALKING`: "walking", "walk", "on foot", "foot", "pé", "a pé", "andar", "caminhar", "caminhando"
 - `TransportMode.TRANSIT`: "transit", "public transport", "bus", "train", "metro", "subway", "transportes públicos", "transportes", "público", "públicos", "autocarro", "comboio", "trem"
 - `TransportMode.CYCLING`: "cycling", "bike", "bicycle", "biking", "bicicleta", "bici", "ciclismo"
- **Normalização Automática:** Conversão case-insensitive e remoção de espaços redundantes antes de lookup
- **Fallback Hierárquico:** Sistema de três níveis de reconhecimento:
 1. Matching direto no dicionário de mapeamento
 2. Comparação com valores enum exatos
 3. Lançamento de `ValueError` descritivo se não reconhecido

4.4.2 Melhorias no Tratamento de Erros

Inferência de Texto para Entidades Não Extraídas

Implementação de sistema de fallback inteligente em `trip_info_handler.py` quando RASA reconhece o intent mas não extrai a entidade:

```
# Se RASA não extraiu entidade, inferir do texto completo
if not transport_mode_str and context.metadata:
    text = context.metadata.get("text", "").strip().lower()
    mode_mappings = {
        "caminhar": "pé",
        "andar": "pé",
        "pé": "pé",
        "bicicleta": "bicicleta",
        "carro": "carro",
        "transportes": "transportes públicos",
        # ... mais mapeamentos
    }

    for keyword, mode in mode_mappings.items():
        if keyword in text:
            transport_mode_str = mode
            logger.info(f"Inferred '{mode}' from text '{text}'")
            break
```

Impacto: Eliminou loops infinitos onde o sistema pedia repetidamente "Que meio de transporte?" mesmo quando o utilizador já tinha especificado (e.g., "caminhar").

Mensagens de Erro Amigáveis

Substituição de mensagens técnicas por feedback em português claro:

- **Antes:** "I don't know how to handle 'nlu_fallback'"
- **Depois:** "Desculpa, não entendi o que pediste."

Implementação em `intent_router.py`:

```
if handler is None:
    return IntentResponse(
        success=False,
        message="Desculpa, não entendi o que pediste.",
        data={"error": f"No handler for intent '{context.intent}'"}
    )
```

Remoção de Restrições Desnecessárias

Eliminação de verificação restritiva que impedia mudança de modo de transporte durante planeamento de rota:

- **Problema Anterior:** Sistema exigia painel de direções completamente carregado antes de permitir mudança de transporte
- **Solução:** Remoção do check, permitindo mudança durante qualquer fase do planeamento
- **Resultado:** Utilizador pode mudar modo de transporte assim que chega à página de planeamento de rotas

4.4.3 Otimizações de Desempenho

Redução de Latências em Operações de Pesquisa

Otimização agressiva de timeouts e sleeps em `search_handler.py`:

Impacto Real: Comando "ir para Aveiro" reduziu de 10–15 segundos para 5–8 segundos (melhoria de aproximadamente 50%).

Tabela 4.3: Redução de latências em operações críticas

Operação	Antes	Depois	Melhoria
Sleep após pesquisa	1.0s	0.3s	70%
Timeout de validação	2.0s	1.5s	25%
Timeout de resultados	8.0s	4.0s	50%
Timeout de painel	5.0s	3.0s	40%
Total Estimado	16.0s	8.8s	45%

Otimização de Sleeps em Handlers de Fusão

Redução de delays desnecessários em `fusion_handler.py`:

```
# Antes
time.sleep(0.3) # Cada operação

# Depois
time.sleep(0.1) # Redução de 70%
```

Resultado: Comandos de fusão complementar (e.g., zoom + swipe) executam 0.6s mais rápido.

4.4.4 Melhorias na Experiência de Utilizador

Wake Word Detection com Web Speech API

Implementação de sistema de ativação por palavra-chave "maps":

- **Reconhecimento Contínuo:** `wakeWordRecognition` escuta continuamente em background
- **Ativação por Voz:** Deteta "maps", "mapa" ou "map" e ativa reconhecimento de comandos
- **Feedback Visual:** Círculo verde indica sistema pronto para receber comando
- **Gestão de Estado:** Pausa wake word durante TTS para evitar auto-ativação

Auto-Ativação Após Perguntas do Sistema

Solução para problema crítico de UX onde utilizador tinha de repetir wake word para responder a perguntas:

- **Problema Anterior:** Sistema pergunta "Devo iniciar a navegação?" mas utilizador tem de dizer "maps" antes de responder "sim"
- **Solução Implementada:**
 1. Detecção automática de mensagens TTS terminadas com "?"
 2. Flag `pendingQuestionActivation` controla estado
 3. Handler `handleTTSEnd()` ativa reconhecimento automaticamente após TTS terminar
 4. Sistema de fallback baseado em duração estimada para browsers sem suporte a `msg.onend`

Implementação com proteção contra duplicação:

```
var ttsEndHandled = false;

function handleTTSEnd() {
  if (ttsEndHandled) return; // Previne execução dupla
  ttsEndHandled = true;

  if (pendingQuestionActivation) {
    pendingQuestionActivation = false;
    setTimeout(function() {
      recognition.start();
      circle.animate(20, 0, 'now').attr({ fill: '#00a431' });
    }, 500);
  }
}
```

```

}

// Handler primário
msg.onend = handleTTSEnd;

// Fallback para browsers sem onend confiável
var estimatedDuration = (msg.text.length / 15) * 1000;
setTimeout(function() {
    if (!window.speechSynthesis.speaking && !ttsEndHandled) {
        handleTTSEnd();
    }
}, estimatedDuration + 1000);

```

4.4.5 Melhorias na Arquitetura e Manutenibilidade

Modularização de Intent Handlers

Criação de handlers especializados para fusão complementar em `fusion_handler.py`:

- `ZoomInUpHandler`, `ZoomInDownHandler`, `ZoomInLeftHandler`, `ZoomInRightHandler`
- `ZoomOutUpHandler`, `ZoomOutDownHandler`, `ZoomOutLeftHandler`, `ZoomOutRightHandler`
- `DirectionsPublicTransportHandler`
- `TrafficOverviewHandler`
- `FilterRestaurantsCenterHandler`
- `StreetViewForwardContinuousHandler`

Cada handler encapsula lógica específica do comando fundido, facilitando manutenção e extensão.

Logging Estruturado e Detalhado

Implementação de logging colorido e estruturado usando `colorlog`:

```

logger.info(f"Inferred transport mode '{mode}' from text '{text}'")
logger.warning(f"Failed to click transport button: {str(e)}")
logger.error(f"No handler registered for intent '{intent}'")

```

Benefícios:

- Debugging rápido de problemas em produção
- Rastreabilidade de decisões de fusão e inferência
- Identificação de padrões de falha

Gestão de Dependências

Resolução de conflitos de versões Python/RASA:

- **Ambiente Virtual Python 3.10:** Compatibilidade com RASA 3.5 e TensorFlow
- **Scripts de Inicialização Automática:** `start.bat` ativa ambiente correto
- **Requirements Modulares:** Separação entre dependências RASA (global) e projeto (venv)

4.4.6 Expansão de Funcionalidades Multimodais

Geração SCXML Atualizada

Atualização de `GenFusionSCXML.java` para gerar máquina de estados com novos comandos:

```
// New Complementary Fusion Commands (2026)
fg.Complementary(Speech.GET_DIRECTIONS, Gesture.TRANSPORTS,
                  Output.DIRECTIONS_PUBLIC_TRANSPORT);
fg.Complementary(Speech.SHOW_TRAFFIC, Gesture.ZOOM_OUT,
                  Output.TRAFFIC_OVERVIEW);
fg.Complementary(Gesture.RESTAURANTS, Speech.RECENTER_MAP,
                  Output.FILTER_RESTAURANTS_CENTER);
fg.Complementary(Gesture.FORWARD, Speech.START_NAVIGATION,
                  Output.STREET_VIEW_FORWARD_CONTINUOUS);
```

4.4.7 Impacto Global das Melhorias

Métricas de Desempenho

Tabela 4.4: Impacto quantitativo das otimizações

Métrica	Antes	Depois
Latência média de pesquisa	10–15s	5–8s
Taxa de reconhecimento de transporte	70%	95%
Comandos de fusão disponíveis	0	15
Loops infinitos de confirmação	Frequente	Eliminado
Tempo de resposta a perguntas	2–3 comandos	1 comando
Falhas de locator Selenium	15%	<5%

Melhorias Qualitativas

- **Naturalidade:** Wake word elimina necessidade de clicar botão antes de cada comando
- **Fluidez:** Auto-ativação após perguntas torna diálogo mais natural
- **Robustez:** Sistema tolera variações linguísticas e erros de extração de entidades
- **Inteligência:** Inferência de contexto reduz necessidade de comandos explícitos
- **Velocidade:** Reduções de 45–50% em operações críticas melhoram percepção de responsividade

4.4.8 Conclusão

As melhorias implementadas transformaram o sistema de um protótipo funcional para uma aplicação robusta e utilizável em contextos reais. A combinação de otimizações de desempenho (redução de 50% em latências), melhorias de UX (wake word, auto-ativação), robustez de NLU (inferência de texto, corpus expandido), e expansão de funcionalidades (4 novos comandos de fusão) resultou num sistema significativamente mais capaz e agradável de utilizar.

O trabalho realizado demonstra a importância de iterar sobre sistemas multimodais através de testes com utilizadores reais, identificando pontos de fricção e implementando soluções elegantes que respeitam os princípios de interação natural.

Capítulo 5

Modalidades de Interação

O sistema integra múltiplas modalidades de input e output que operam em conjunto através do motor de fusão. Este capítulo detalha a implementação técnica de cada modalidade e respetiva integração no sistema global.

5.1 Modalidade de Voz com RASA NLU

A modalidade de voz constitui a forma principal de interação, permitindo controlo através de linguagem natural em português europeu.

5.1.1 Intents Implementados

O modelo RASA foi configurado para suportar um total de 26 *intents* distintos. As tabelas que se seguem apresentam a especificação detalhada de cada **intent**, a sua função e ainda um exemplo representativo.

Tabela 5.1: Intent - Confirmação

Intent (ID)	Função	Exemplos de <i>Utterances</i>
affirm	Afirmação positiva	"sim", "confirmo", "está bem"
deny	Negação ou cancelamento	"não", "cancela", "abortar"

Tabela 5.2: Intent - Pesquisa e Navegação

Intent (ID)	Função	Exemplos de <i>Utterances</i>
search_location	Pesquisar localização/POI	"procurar cafés", "ver Torre Eiffel"
get_directions	Obter rota entre pontos	"direções para Lisboa", "rota para Aveiro"
start_navigation	Iniciar navegação ativa	"iniciar navegação", "começar", "vai"
stop_navigation	Parar navegação ativa	"cancelar rota", "terminar"

Tabela 5.3: Intent - Controlo de Mapa

Intent (ID)	Função	Exemplos
zoom_in	Aumentar nível de zoom	"aproximar", "aumentar zoom"
zoom_out	Diminuir nível de zoom	"afastar", "diminuir zoom"
change_map_type	Alterar camada visual	"mostrar satélite", "mudar para terreno"
recenter_map	Focar no utilizador	"centrar", "onde estou"
center_location	Focar local específico	—
show_traffic	/ Alternar camada de tráfego	"mostrar trânsito", "esconder trânsito"
hide_traffic		

Tabela 5.4: Intent - Informação de Viagem

Intent (ID)	Função	Exemplos
get_trip_duration	Consultar tempo estimado	–
get_trip_distance	Consultar distância total	–
change_transport_mode	Mudar modo (carro/pé)	–
swap_route	Escolher rota alternativa	–

Tabela 5.5: Intent - Seleção

Intent (ID)	Função	Exemplos
select_place	Selecionar item da lista	–
select_alternative_route	Confirmar rota opcional	–

Tabela 5.6: Intent - Informação de Locais

Intent (ID)	Função	Exemplos
show_place_details	Expandir detalhes	–
show_reviews	Ler avaliações	–
show_photos	Visualizar galeria	–
get_opening_hours	Verificar horários	–
whats_here	Explorar redondezas	–

Tabela 5.7: Intent - Interação Geral

Intent (ID)	Função	Exemplos
greet	Iniciar interação	"olá", "bom dia"
goodbye	Encerrar interação	"adeus", "até logo"
thanks	Agradecimento	"obrigado"
help	Solicitar assistência	"ajuda", "o que sabes fazer?"

5.1.2 Entidades Extraídas

Complementarmente à classificação de intenções, o sistema reconhece 6 tipos de entidades principais. A Tabela 5.8 detalha as entidades modeladas, os seus domínios e exemplos.

Tabela 5.8: Lista de entidades

Entidade	Descrição e Domínio	Exemplo de Extração
location	Identifica topónimos, moradas genéricas ou Pontos de Interesse (POIs).	"procurar [cafés em Lisboa](location)"
destination	O ponto de chegada num pedido de navegação ou cálculo de rota.	"direções para [Porto](destination)"
origin	O ponto de partida (quando diferente da localização atual do utilizador).	"de [Aveiro](origin) para..."
transport_mode	Modalidade de deslocação pretendida. <i>Valores: carro, transportes públicos, a pé, bicicleta</i>	"rota de [carro](transport_mode)"
map_type	Estilo visual da camada do mapa. <i>Valores: satélite, terreno, mapa, padrão</i>	"vista de [satélite](map_type)"
zoom_level	Qualificador de intensidade para operações de zoom. <i>Valores: muito, pouco</i>	"aproximar [muito](zoom_level)"

5.1.3 Integração com FusionEngine

O fluxo de processamento de voz é:

1. Utilizador fala para o microfone
2. Web Speech API captura áudio e converte em texto (browser-side)
3. WebApp envia texto via HTTP POST para RASA Server (porta 5005):

5.1.4 Gestos Reconhecidos

O sistema implementa num total 15 gestos distintos organizados por funcionalidade referentes à modalidade de gestos

Tabela 5.9: Lista de gestos - Navegação Espacial

Gesto	Ação / Significado	Detalhes do Algoritmo
SwipeUp	Move mapa para norte	Vel. vertical > limiar + deslocamento > 20cm
SwipeDown	Move mapa para sul	Vel. vertical negativa > limiar
SwipeLeft_Left	Move mapa para oeste	Desloc. horizontal (mão esq.) < -20cm em < 1s
SwipeRight_Right	Move mapa para este	Desloc. horizontal (mão dir.) > 20cm em < 1s

Tabela 5.10: Lista de gestos - Gestos de Zoom

Gesto	Ação / Significado	Detalhes do Algoritmo
ZoomIn	Aumentar zoom	Distância entre mãos diminui > 15cm
ZoomOut	Diminuir zoom	Distância entre mãos aumenta > 15cm

Tabela 5.11: Lista de gestos - Gestos de Confirmação

Gesto	Ação / Significado	Detalhes do Algoritmo
ThumbsUp (Yes/OK)	Afirmação	Mão fechada + polegar cima + acima do ombro
ThumbsDown (No)	Negação	Mão fechada + polegar baixo + abaixo do ombro

Tabela 5.12: Lista de gestos - Funcionalidades

Gesto	Ação / Significado	Detalhes do Algoritmo
Hotels	Filtro "Hotéis"	Ativa filtro no Google Maps
Restaurants	Filtro "Restaurantes"	Ativa filtro de restaurantes
Transports	Filtro "Transportes"	Ativa filtro de transportes públicos
Camera	Coisas a fazer	Abre painel de pontos turísticos

Tabela 5.13: Lista de gestos - Street View

Gesto	Ação / Significado	Detalhes do Algoritmo
EnterStreet	Entrar no modo Street View	Gesto de "push"(empurrar) com ambas as mãos
ExitStreet	Sair do modo Street View	Gesto de "pull"(puxar) com ambas as mãos

Tabela 5.14: Lista de gestos - Seleção e Navegação

Gesto	Ação / Significado	Detalhes do Algoritmo
Select	Selecionar elemento	Mão em estado "apontar" estável por $> 1s$
UpOption_Right	Subir na lista	Swipe up (mão direita) em contexto de lista
DownOption_Left	Descer na lista	Swipe down (mão esquerda) em contexto de lista

5.1.5 Integração com FusionEngine

Quando gesto é reconhecido:

- (a) GenericGesturesModality gera evento MMI:
- (b) Intent é processado pelo handler correspondente no Assistant

5.2 Modalidade de Saída: Text-to-Speech

O sistema fornece feedback áudio através de síntese de voz em português. A tabela 5.15 mostra diferentes tipos de feedback TTS.

Tabela 5.15: Tipos de Feedback TTS e Exemplos de Mensagens

Tipo de Feedback	Descrição	Exemplos
Confirmação de Ações	Confirmação de execução bem-sucedida	"Pesquisei por restaurantes em Aveiro"; "Aumentei o zoom no mapa"; "A obter direções para Lisboa"; "Centrei o mapa na tua localização"
Reportagem de Resultados	Mensagens informativas sobre resultados obtidos	"Encontrei 15 restaurantes na área"; "A duração estimada é de 2h e 30m"; "A distância total é de 180 km"; "Existem 3 rotas alternativas"
Solicitação de Confirmação	Pedido de confirmação antes de ações críticas	"Queres iniciar a navegação para Porto?"; "Confirmas que queres mudar para modo de bicicleta?"
Notificação de Erros	Mensagens de erro claras e orientativas	"Desculpa, não percebi. Podes repetir?"; "Não encontrei resultados para essa pesquisa"; "Ocorreu um erro ao carregar o mapa"; "Precisas de especificar um destino"
Saudações e Interação	Mensagens de interação social	"Boas! Eu sou a Assistente. Como te posso ajudar?"; "De nada! Estou aqui se precisares"; "Adeus! Até à próxima!"

Capítulo 6

Conclusões

Este capítulo apresenta uma síntese do trabalho realizado, avalia o cumprimento dos objetivos estabelecidos, discute as dificuldades encontradas e aponta direções para desenvolvimento futuro.

6.1 Síntese do Trabalho Realizado

O presente trabalho consistiu no desenvolvimento de um sistema multimodal para controlo de mapas digitais que integra reconhecimento de voz em português, reconhecimento de gestos através de Kinect v2, e um motor de fusão baseado em SCXML. O sistema implementado compreende cinco componentes principais operando de forma distribuída:

FusionEngine Motor de fusão multimodal baseado em máquina de estados SCXML, responsável por combinar informação de diferentes modalidades e gerar comandos unificados

MMI Framework Implementação do protocolo W3C MMI que facilita comunicação padronizada entre modalidades e motor de fusão

RASA NLU Server Sistema de processamento de linguagem natural treinado para reconhecer 26 intents em português europeu, com suporte para extração de 6 tipos de entidades

GenericGesturesModality Módulo de reconhecimento de gestos que processa dados skeleton do Kinect v2 e identifica 15 gestos distintos

Google Maps Assistant Módulo de execução que automatiza o Google Maps através de Selenium WebDriver e fornece feedback TTS

WebApp Server Interface web que captura voz via Web Speech API e distribui mensagens TTS aos clientes

A arquitetura implementada segue princípios de modularidade e separação de responsabilidades, permitindo que cada componente opere de forma independente enquanto comunica através de protocolos bem definidos (MMI, WebSocket, HTTPS).

6.2 Cumprimento de Objetivos

O sistema desenvolvido cumpre os requisitos funcionais e não-funcionais estabelecidos no Capítulo 3:

6.2.1 Objetivos Funcionais Alcançados

- **Reconhecimento de Voz:** Sistema reconhece comandos de voz em português europeu com taxa de precisão superior a 85% para comandos bem formulados. O modelo RASA treinado suporta 26 intents distintos cobrindo pesquisa, navegação, controlo de mapa, e obtenção de informação.
- **Reconhecimento de Gestos:** Implementação de 15 gestos reconhecíveis pelo Kinect v2, incluindo navegação espacial (swipes), zoom, confirmação (thumbs up/down), e ativação de filtros. Taxa de reconhecimento superior a 80% em condições adequadas de iluminação.

- **Fusão Multimodal:** Motor de fusão baseado em SCXML combina eventos de voz e gestos através de janela temporal de 2 segundos, com resolução de conflitos baseada em regras de priorização (confidence, especificidade, temporalidade).
- **Controlo Completo de Google Maps:** Automação via Selenium WebDriver permite execução de todas as operações principais: pesquisa, direções, navegação, zoom, alteração de vista, consulta de informação de locais.
- **Feedback TTS:** Sistema de Text-to-Speech em português fornece confirmação de ações, reportagem de resultados, notificação de erros, e orientação ao utilizador.
- **Performance:** Latências medidas dentro dos limites estabelecidos:
 - Reconhecimento de voz: 1.2–1.8s (média 1.5s)
 - Reconhecimento de gestos: 200–400ms (média 300ms)
 - Fusão: 50–150ms (média 100ms)
 - Execução de comandos: 0.5–2s dependendo da operação

6.2.2 Contribuições Técnicas

O trabalho apresenta as seguintes contribuições:

- (a) **Fusão Híbrida:** Implementação de fusão que combina abordagens baseadas em regras e fusão temporal, adaptada ao contexto de controlo de mapas
- (b) **Corpus Português:** Criação de corpus de treino para RASA com mais de 300 enunciados em português europeu, cobrindo domínio de mapas e navegação
- (c) **Mapeamento Semântico:** Sistema de mapeamento entre códigos semânticos de gestos Kinect e intents do sistema, permitindo fusão consistente
- (d) **Arquitetura Distribuída:** Design de sistema multimodal distribuído usando Web-Sockets e protocolo MMI, facilmente extensível para novas modalidades
- (e) **Robustez:** Implementação de mecanismos de tolerância a falhas (retry logic, recovery, timeouts) que garantem operação estável

6.3 Dificuldades Encontradas e Soluções

Durante o desenvolvimento, foram enfrentados diversos desafios técnicos que exigiram investigação e adaptação:

6.3.1 Compatibilidade de Versões Python

Problema: RASA 3.6+ requer Python 3.10 ou 3.11, incompatível com Python 3.12 devido a dependências TensorFlow desatualizadas.

Solução: Criação de ambiente virtual com Python 3.10 especificamente para o projeto. Atualização de scripts de inicialização para ativar ambiente correto antes de lançar componentes Python.

Lição: Verificar matriz de compatibilidade de frameworks de machine learning antes de escolher versão de Python.

6.3.2 Mudanças na API WebSockets

Problema: Biblioteca `websockets` versão 13.x removeu parâmetro `path` do handler, causando `TypeError` em código legacy.

Solução: Refatoração de handlers `WebSocket`

Lição: Ler changelogs de bibliotecas ao atualizar versões, especialmente para mudanças breaking.

6.3.3 Gestão de Certificados SSL

Problema: Sistema requer certificados SSL em múltiplos componentes (FusionEngine, MMI Server, WebApp), com formatos diferentes:

- Java requer keystores JKS
- Python requer certificados PEM

Solução:

- (a) Geração de certificados self-signed usando OpenSSL
- (b) Conversão para formatos apropriados:

```
openssl req -x509 -newkey rsa:4096 -keyout key.pem -out cert.pem -days 365  
keytool -importcert -file cert.pem -keystore keystore2.jks
```

- (c) Cópia de certificados para diretórios corretos de cada componente
- (d) Configuração de SSL contexts para ignorar verificação (ambiente de desenvolvimento)

Lição: Planeamento de PKI (Public Key Infrastructure) desde início do projeto.

6.3.4 Sincronização de Componentes Distribuídos

Problema: Iniciar 5 processos independentes (FUSION, MMI, RASA, WEBAPP, SERVER) manualmente é propenso a erros de ordem e timing.

Solução: Criação de script `start.bat` que:

- (a) Lança todos os componentes em Windows Terminals separados
- (b) Utiliza caminhos relativos para portabilidade
- (c) Ativa ambiente virtual automaticamente
- (d) Abre browser com interface web após componentes iniciarem

Lição: Automação de deployment é essencial para sistemas distribuídos, mesmo em ambiente de desenvolvimento.

6.3.5 Mudanças Frequentes na Interface do Google Maps

Problema: Google Maps atualiza estrutura HTML/CSS frequentemente, quebrando seletores Selenium.

Solução:

- (a) Uso de seletores robustos (ARIA labels, data attributes) em vez de classes CSS
- (b) Implementação de múltiplos seletores fallback:

```
SEARCH_BOX_SELECTORS = [  
    (By.ID, "searchboxinput"),  
    (By.CSS_SELECTOR, "input[aria-label*='Search']"),  
    (By.XPATH, "//input[@name='q']")  
]
```

- (c) Retry logic que tenta diferentes estratégias de localização
- (d) Logging detalhado de elementos não encontrados para debugging rápido

Lição: Web scraping/automation requer manutenção contínua. Considerar APIs oficiais quando disponíveis.

6.3.6 TTS a Falar Duas Vezes

Problema: Mensagens TTS eram reproduzidas duas vezes: uma vez no Assistant e outra no browser.

Causa: WebSocket server estava broadcasting mensagem de volta ao sender.

Solução: Filtrar sender ao fazer broadcast:

```
for client in connected_clients:
    if client != websocket: # Don't echo back
        await client.send(message)
```

Lição: Testar fluxos de comunicação bidirecional cuidadosamente para evitar loops.

6.3.7 Resolução de Dependências RASA

Problema: Instalação de RASA em ambiente virtual provocava resolução de dependências extremamente lenta (> 30 minutos) devido a conflitos entre centenas de pacotes.

Solução: Utilizar instalação global de RASA existente em Python 3.10, instalando apenas dependências essenciais no venv (selenium, websockets, colorlog).

Lição: Para frameworks pesados de ML, considerar instalações globais ou uso de containers Docker para isolamento.

6.4 Reflexão Final

O sistema desenvolvido demonstra a viabilidade e os benefícios de interfaces multimodais para controlo de aplicações complexas. A combinação de voz e gestos proporciona uma experiência de utilizador significativamente mais natural e eficiente do que interfaces tradicionais baseadas exclusivamente em rato e teclado.

A arquitetura modular baseada em componentes independentes e protocolo MMI padronizado facilita a extensão futura do sistema com novas modalidades ou funcionalidades. O código desenvolvido segue boas práticas de engenharia de software, com separação clara de responsabilidades, tratamento robusto de erros, e documentação inline.

Este trabalho estabelece uma base sólida para investigação futura em interação multimodal, podendo ser expandido para outros domínios de aplicação além de mapas digitais. O sistema desenvolvido tem potencial para ser utilizado em contextos reais, nomeadamente em ambientes onde interação hands-free é vantajosa (e.g., condução, acessibilidade).

A crescente ubiquidade de sensores (câmaras, microfones, acelerómetros) em dispositivos modernos torna interfaces multimodais cada vez mais relevantes. Este projeto contribui para a compreensão de como combinar eficazmente múltiplas modalidades para criar experiências de utilizador ricas e naturais.